

Chapter 6: Tuples

Study Notes

In Chapter 5 you learned that lists are **mutable** -- you can change an element after creation simply by writing `fruits[1] = "blueberry"`. A **tuple** looks almost identical to a list and supports most of the same operations, but it draws a hard line in the sand: once created, a tuple can never be changed. This single difference -- mutable vs. immutable -- is what this entire chapter is built around.

What you will learn in this chapter:

- Creating tuples, indexing, slicing, and `len()` -- all reused from Chapters 4 and 5
- The single-element tuple syntax quirk: `(5)` vs `(5,)`
- Tuple immutability, directly contrasted with list mutability
- Tuple packing and unpacking, including starred `(*)` unpacking
- Tuple methods: `count()` and `index()`
- The `in` / `not in` membership operators on tuples
- Tuple vs. List -- when to use which

1. Creating Tuples and Accessing Elements

A **tuple** is an ordered collection of values, usually written inside parentheses `( )` with values separated by commas. Parentheses are actually optional -- it is the **COMMAS** that make a tuple, not the parentheses -- but writing them is standard practice because it makes the tuple easy to spot.

Example

```
point = (10, 20, 30)
no_parens = 10, 20, 30      # still a tuple -- the commas matter

print(point)
print(no_parens)
print(type(point))
print(type(no_parens))
```

Output

```
(10, 20, 30)
(10, 20, 30)
<class 'tuple'>
<class 'tuple'>
```

Tuples are sequences, just like the strings from Chapter 4 and the lists from Chapter 5, so indexing, `len()`, and slicing all work exactly the same way.

Example

```
colors = ("red", "green", "blue", "yellow")

print(colors[0])      # first element
print(colors[-1])     # last element
print(len(colors))    # number of elements
print(colors[1:3])    # slicing: index 1 up to (not including) 3
print(colors[::-1])   # the whole tuple, reversed
```

Output

```
red
yellow
4
('green', 'blue')
('yellow', 'blue', 'green', 'red')
```

Reused from earlier chapters

Indexing with `[]` (positive and negative), the `len()` function, and slice notation `[start:stop:step]` behave identically on strings, lists, and tuples. If you are comfortable with Chapters 4 and 5, this part of tuples requires no new mental model at all.

2. The Single-Element Tuple Quirk

Python uses parentheses for two completely different purposes: grouping a mathematical expression, AND building a tuple. This creates a surprising trap when you try to make a tuple with only ONE element.

```
a = (5)
b = (5,)

print(type(a))
print(type(b))
```

Output

```
<class 'int'>
<class 'tuple'>
```

`(5)` is just the integer 5 wrapped in ordinary (mathematical) parentheses -- Python evaluates it as plain 5. It is the trailing COMMA in `(5, )`, not the parentheses, that actually creates a tuple. This is one

of the most common beginner mistakes with tuples (see Section 8).

Rule of thumb

If you want a tuple with a single item, you **MUST** include a trailing comma: `single = ("only_item",)`. Forgetting it silently gives you back the original value's type instead of a tuple, with no error raised -- which makes this mistake easy to miss.

3. Tuple Immutability

Recall from Chapter 5 that lists are mutable: `fruits[1] = "blueberry"` works perfectly. Tuples are the opposite -- once created, **NONE** of their elements can be changed, added, or removed. Attempting to do so raises a `TypeError`, the same way modifying a string by index did in Chapter 4.

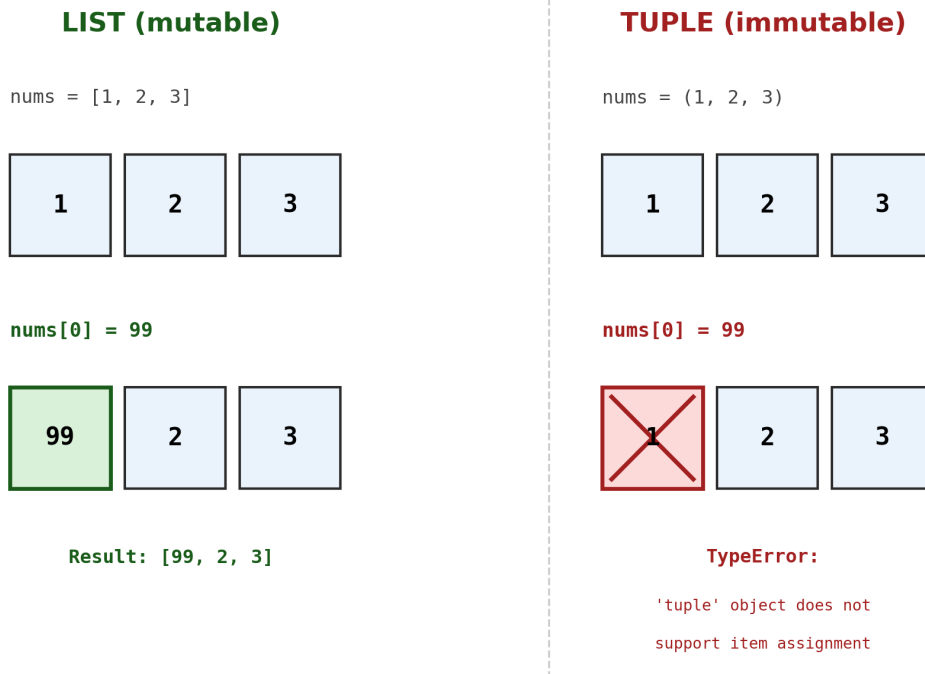


Figure 1: The same assignment, `nums[0] = 99`, succeeds on a list but raises a `TypeError` on a tuple.

This will raise an error:

```
nums = (1, 2, 3)
nums[0] = 99          # attempting to change the first element
```

Output

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    nums[0] = 99
TypeError: 'tuple' object does not support item assignment
```

Compare this directly with the equivalent list code from Chapter 5, which runs without any error:

```
nums = [1, 2, 3]
nums[0] = 99          # this is FINE -- lists are mutable
print(nums)
```

Output

```
[99, 2, 3]
```

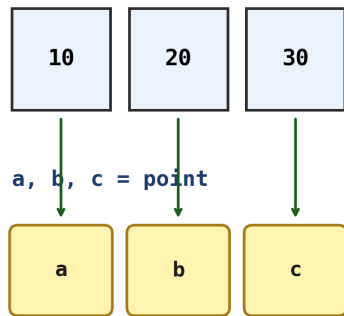
Why use something you can't change?

Immutability is a feature, not a limitation. A tuple signals to anyone reading your code -- including future you -- that this collection of values is meant to stay fixed, such as a pair of (latitude, longitude) coordinates or the (red, green, blue) channels of a color. Section 7 explores this trade-off in more detail.

4. Tuple Packing and Unpacking

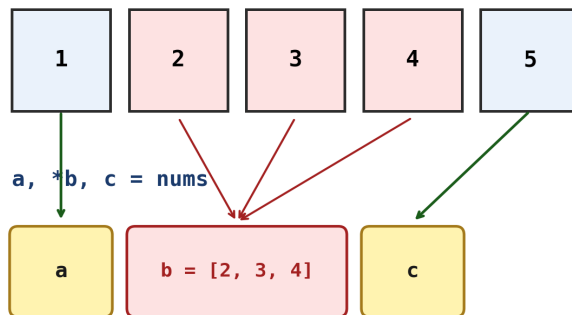
"Packing" simply means grouping several values into a tuple, which you have already been doing throughout this chapter. "Unpacking" is the reverse: assigning a tuple's elements to several variables at once, in a single line.

```
point = (10, 20, 30)
```



each variable receives exactly one value, in order

```
nums = (1, 2, 3, 4, 5)
```



** collects all the "leftover" middle items into a list*

Figure 2: Top -- each variable receives exactly one value from the tuple, in order. Bottom -- a starred variable (`*b`) scoops up every "leftover" element into a list.

Basic unpacking

```
point = (10, 20, 30)
a, b, c = point

print(a)
print(b)
print(c)
```

Output

```
10
20
30
```

The number of variables on the left must match the number of elements in the tuple, or Python raises a `ValueError` (see Section 8).

Unpacking with * for extra elements

Sometimes you only care about the first item, the last item, or some fixed number of items, and want everything else collected together. Prefixing one variable with `*` tells Python to scoop up all remaining values into a `LIST`, no matter how many there are.

```
nums = (1, 2, 3, 4, 5)
a, *b, c = nums

print(a)
print(b)
print(c)
```

Output

```
1
[2, 3, 4]
5
```

Note the type

The starred variable always ends up as a `LIST`, even though it was unpacked from a tuple. In the example above, `b` is `[2, 3, 4]` -- a list, not a tuple.

5. Tuple Methods: `count()` and `index()`

Because tuples cannot be changed, they support far fewer methods than lists -- there is no `append()`, `insert()`, `remove()`, `pop()`, `sort()`, or `reverse()`, since all of those would require modifying the tuple in place. Only two methods remain, and both simply look at the existing data without changing it.

5.1 `count(value)`

Returns how many times a value appears in the tuple.

```
nums = (1, 2, 2, 3, 2, 4)
print(nums.count(2))
```

Output

```
3
```

5.2 `index(value)`

Returns the index of the **FIRST** occurrence of a value. If the value is not present, it raises a `ValueError` (this is different from the string method `find()` from Chapter 4, which quietly returns `-1` instead of raising an error).

```
nums = (1, 2, 2, 3, 2, 4)
print(nums.index(2))
print(nums.index(3))
```

Output

```
1
3
```

6. Membership: in and not in

The `in` and `not in` operators work on tuples exactly as they did on strings in Chapter 4 and lists in Chapter 5 -- they check whether a value exists in the sequence and return a boolean.

```
colors = ("red", "green", "blue")

print("green" in colors)
print("purple" in colors)
print("purple" not in colors)
```

Output

```
True
False
True
```

7. Tuple vs. List: When to Use Which

Lists and tuples can both store an ordered collection of values, and most beginners' first instinct is to reach for a list every time. The table below summarizes the practical differences that should guide your choice.

Aspect	List	Tuple
Syntax	Square brackets: <code>[1, 2, 3]</code>	Parentheses: <code>(1, 2, 3)</code>
Mutability	Mutable -- can change, add, remove elements	Immutable -- fixed once created
Available methods	Many: <code>append</code> , <code>insert</code> , <code>remove</code> , <code>pop</code> , <code>sort</code> , ...	Only two: <code>count</code> , <code>index</code>
Typical use case	A collection that will grow, shrink, or change	A fixed collection, e.g. coordinates, RGB values, record-like data
Performance	Slightly more overhead (supports resizing)	Slightly faster and more memory-efficient

Safety	Can be modified by accident or by other code	Cannot be modified -- safer for fixed data
---------------	--	--

Example scenarios

- **Use a list** for a shopping cart that items get added to and removed from, a queue of tasks to process, or any collection you expect to grow or shrink while the program runs.
- **Use a tuple** for a (latitude, longitude) coordinate pair, the (year, month, day) of a fixed date, RGB color values (255, 0, 0), or any small fixed-size record where the meaning of each position matters and should never accidentally change.

```
# A list: this cart is expected to change as the user shops
cart = ["bread", "milk"]
cart.append("eggs")

# A tuple: this coordinate should never be modified
location = (26.9855, 75.8513) # (latitude, longitude)
```

8. Common Beginner Mistakes

Mistake 1: Forgetting the trailing comma for a single-element tuple

(5) is just the integer 5; (5,) is a one-element tuple. No error is raised when you forget the comma, so this mistake can silently break code that expects a tuple later on.

Mistake 2: Trying to modify a tuple element

nums[0] = 99 raises `TypeError: 'tuple' object does not support item assignment`. If you need a "changed" version, you must build an entirely new tuple, just as you would rebuild an immutable string in Chapter 4.

Mistake 3: Confusing when to use a tuple vs. a list

Using a list for data that should never change (like a fixed coordinate) leaves it open to accidental modification elsewhere in the code. Using a tuple for data you intend to grow forces you to rebuild a new tuple every time, which is wasteful. Match the container to whether the data is expected to change -- see the comparison table in Section 7.

Mistake 4: Unpacking errors from a variable-count mismatch

If the number of variables on the left does not match the number of elements in the tuple, Python raises a `ValueError`. For example, `a, b = (1, 2, 3)` fails because there are 3 values but only 2 variables to receive them.

```
point = (1, 2, 3)
a, b = point      # ValueError: too many values to unpack
```

Output


```
ValueError: too many values to unpack (expected 2)
```

If you are not sure how many elements there are, or only need some of them, reach for starred unpacking (Section 4) instead of guessing the exact count.

Chapter 6 Recap

- Tuples are written with parentheses (commas are what actually matter); indexing, `len()`, and slicing all work the same way they did on strings and lists.
- `(5)` is an int; `(5,)` is a one-element tuple -- the trailing comma is required.
- Tuples are IMMUTABLE: `nums[0] = 99` raises a `TypeError`, unlike the equivalent list assignment from Chapter 5.
- Packing groups values into a tuple; unpacking assigns them to several variables at once; `a, *b, c = ...` collects extra elements into a list.
- `count()` and `index()` are the only two tuple methods, since tuples cannot be modified.
- `in` and `not in` test for membership on tuples just as they did on strings and lists.
- Use a list for data that changes; use a tuple for fixed, record-like data that should stay constant.

Looking ahead to Chapter 7: Sets

Chapter 7 introduces sets, another collection type -- but one with no fixed order and no indexing at all, where every element is guaranteed to be unique. This is a useful contrast: lists and tuples both preserve the order you put values in (and allow duplicates), while sets give up ordering in exchange for automatic duplicate removal and fast membership checks.